

Notes (Week 8 Monday)

These are the notes I wrote during the lecture.

DFA (deterministic finite automata)

NFA (non-deterministic finite automata)

In common:

- a finite set of states
- a transition relation between the states, labelled with symbols from an alphabet Σ
- what they do is accept or reject words
 - there's a starting state, and a set of final state
- a word is accepted iff there is a sequence of transitions that consumes the word, leading from the initial to a final state.

In DFA:s the transition relation is a function for each state and label, there's exactly one outgoing transition.

In NFA:s:

- the transition relation is a relation
 - For each state and label, there's 0 or more outgoing transitions
- you may have transitions with *no* label. these can be taken without consuming a character

Show that regular languages are closed under various operators (complement, union, ...)

To that, we can do one of two things:

- show how to build an NFA for e.g. $A \cup B$, given NFA:s for A and B
- show how to build a DFA for e.g. $A \cup B$, given DFA:s for A and B

Complement is regular because,
for any DFA with language L,

if we invert the set of final states,
we get a DFA the accepts L^C

```
void main() {
```

```
}
```

```
[void,main,(,),{,}] <-- list of *tokens*
```

```
L(010 + 101) =
L(010) ∪ L(101) =
L(0)·(L(10)) ∪ L(1)·L(01) =
... =
{0}·{1}·{0} ∪ {1}·{0}·{1} =
{010, 101}
```

```
resumption: 11:23
```

```
// 3 + 2 = 8
x = 3;
// x + 2 = 8
y = x + 2
// y = 8
```

Not all programming is structured around
manipulating state (imperative programming)
Other paradigms exist

Declarative programming,
Functional programming
^ the program is specified as a *function*
that given *values* as input,
computes an output value

```
f(0) = 0
f(n) = n + f(n-1)
^in functional programming languages,
code tends to look like that
```

skip a statement that does nothing
 does nothing = doesn't change
 the state

```
x := x
```

if statements look odd

if g then	if(g){
P	P
else	} else {
Q	Q
fi	}

we're forced to write the else clause

if you want to write a program without
an else clause

if g then P else skip fi

abort a program that always fails
 never terminates successfully

while true do skip od

^ this encoding assumes that we don't
care about the difference between
failure ("the program crashes")
divergence ("the program runs forever")

$\{\phi\}$ P $\{\psi\}$
pre prog post

A Hoare triple is a *specification* of the
intended behaviour of P.

If the initial state
satisfies the precondition ϕ ,
and if P terminates,
then the final state satisfies ψ .

^ note: this is the same thing as
partial correctness from w7

```
// 3 + 2 = 8
x = 3;
y = x + 2
// y = 8
```

^ what I wrote represents a Hoare triple!

$\{3 + 2 = 8\}$ x:=3; y:=x+2 $\{y=8\}$